

Vistas y permisos

Academia Código Norte · cada persona ve solo lo que necesita

Duración	Qué vas a aprender	Necesitas
1–2 sesiones (~1,5 h)	REVOKE, CREATE VIEW, roles, usuarios y GRANT	Haber terminado la práctica 5

Dónde estamos

La base de datos está completa (es el modelo de la práctica 5). Ahora la vamos a usar como en una academia de verdad: la **repcionista**, Lucía, da de alta alumnos y pagos, pero no tiene por qué ver su fecha de nacimiento ni tocar notas. El **profesor**, Andrés, solo ve y pone notas.

La idea se llama **principio de mínimo privilegio**: cada persona puede hacer justo lo que necesita para su trabajo, y nada más.

Cómo trabajar hoy

Igual que en las prácticas anteriores: **una consulta guardada por cada bloque de código**, con estos nombres: P6 · 1 · Revoke API, P6 · 2 · Vista recepción, P6 · 3 · Vista profesor, P6 · 4 · Revoke vistas, P6 · 5 · Roles, P6 · 6 · Permisos, P6 · 7 · Usuarios.

Ejecuta los pasos **en orden** y comprueba que sale «Success» antes de pasar al siguiente.

Las pruebas del paso 5, en una última consulta: P6 · 8 · Pruebas de permisos. Ejecútalas **una a una** seleccionándolas.

Paso 1 · Cierra la puerta de la API

Supabase tiene truco: todo lo que creas en **public** se lo **regala** a **anon** y **authenticated**, que son los usuarios que usa su API. Tal y como está ahora, cualquiera con la clave pública de tu proyecto podría leer los DNI de los alumnos. Lo primero es quitarles todos los permisos. Este es el código:

```
REVOKE ALL ON profesor, alumno, aula, curso, horario, matricula, pago
FROM anon, authenticated;
```

Ojo en Supabase

Quita permisos solo sobre **tus** tablas. No hagas nunca **REVOKE** sobre el esquema **public** ni sobre la base de datos: los usan los servicios internos de Supabase y podrías romper el proyecto.

Paso 2 · Las vistas

Una **vista** es una consulta guardada con nombre que se usa como si fuera una tabla. Así podemos enseñar solo **algunas columnas**. Siempre tiene esta forma:

```
CREATE VIEW nombre_vista AS
SELECT columna1, columna2, ...
FROM tabla;
```

Vista 1 · v_recepcion_alumnos. Es la tabla **alumno** pero solo con algunas columnas: la fecha de nacimiento y el teléfono del tutor se quedan fuera.

```
CREATE VIEW v_recepcion_alumnos AS
SELECT id_alumno, dni, nombre, apellidos, email, telefono, activo
FROM alumno;
```

Vista 2 · v_profesor_notas. Junta datos de tres tablas con **JOIN**, que veremos en el capítulo 3. De momento basta con que entiendas qué muestra: el nombre del curso, el alumno y su nota.

```
CREATE VIEW v_profesor_notas AS
SELECT c.nombre AS curso,
       a.apellidos || ', ' || a.nombre AS alumno,
       m.nota_final
FROM matricula m
JOIN alumno a ON a.id_alumno = m.id_alumno
JOIN curso c ON c.id_curso = m.id_curso;
```

Las vistas nuevas también se las regala Supabase a la API. Quítaselas:

```
REVOKE ALL ON v_recepcion_alumnos, v_profesor_notas FROM anon, authenticated;
```

Comprueba

SELECT * FROM v_recepcion_alumnos; devuelve los 4 alumnos sin fecha de nacimiento.

SELECT * FROM v_profesor_notas; devuelve 5 filas, todas sin nota.

Paso 3 · Los roles y sus permisos

Un **rol** es un «puesto de trabajo» con sus permisos. Primero se crean los dos roles:

```
CREATE ROLE rol_recepcion NOLOGIN;
CREATE ROLE rol_profesor NOLOGIN;
```

Después se reparten los permisos con **GRANT**. Se lee así: **GRANT** permiso **ON** objeto **TO** rol. Código completo:

```
GRANT SELECT ON v_recepcion_alumnos TO rol_recepcion;
GRANT SELECT ON curso, aula, horario TO rol_recepcion;
GRANT INSERT ON alumno, matricula, pago TO rol_recepcion;

GRANT SELECT ON v_profesor_notas TO rol_profesor;
GRANT SELECT (id_alumno, id_curso, nota_final) ON matricula TO rol_profesor;
GRANT UPDATE (nota_final) ON matricula TO rol_profesor;
```

Qué permite cada línea:

Rol	Permiso	Sobre	Para qué
rol_recepcion	SELECT	v_recepcion_alumnos	Ver los alumnos (sin fecha de nacimiento)
rol_recepcion	SELECT	curso, aula, horario	Informar de cursos, aulas y horarios
rol_recepcion	INSERT	alumno, matricula, pago	Dar de alta alumnos, matrículas y pagos
rol_profesor	SELECT	v_profesor_notas	Ver las notas
rol_profesor	SELECT (id_alumno, id_curso, nota_final)	matricula	Leer solo esas 3 columnas
rol_profesor	UPDATE (nota_final)	matricula	Poner notas, y nada más

Paso 4 · Los usuarios

Ahora creamos a las personas y les damos su rol. Copia y ejecuta:

```
CREATE ROLE lucia_recepcion LOGIN PASSWORD 'Recepcion#2026';
CREATE ROLE andres_profesor LOGIN PASSWORD 'Profe#2026';
GRANT rol_recepcion TO lucia_recepcion;
GRANT rol_profesor TO andres_profesor;
```

```
-- Solo en Supabase: para poder hacer las pruebas con SET ROLE
GRANT lucia_recepcion, andres_profesor TO postgres;
```

¿Y esa última línea?

En el SQL Editor trabajas como el usuario `postgres`, que en Supabase no es el jefe absoluto. Para poder «ponerte en la piel» de Lucía o de Andrés con `SET ROLE`, antes tiene que ser miembro de ellos.

Paso 5 · Ponlo a prueba

Con `SET ROLE` te conviertes en otro usuario sin cerrar sesión, y con `RESET ROLE` vuelves a ser tú. Ejecuta las sentencias **una a una** y anota qué pasa. Los comentarios te dicen lo que debería ocurrir (✓ funciona, ✗ da error):

```
SET ROLE lucia_recepcion;
SELECT * FROM v_recepcion_alumnos;                -- ✓
INSERT INTO alumno (id_alumno, dni, nombre, apellidos, fecha_nacimiento, email)
VALUES (5, '99999999J', 'Carla', 'Ruiz Pons', '2006-08-08',
        'carla.ruiz@mail.com');                    -- ✓
SELECT fecha_nacimiento FROM alumno;               -- ✗
UPDATE matricula SET nota_final = 9 WHERE id_curso = 3; -- ✗
RESET ROLE;

SET ROLE andres_profesor;
SELECT * FROM v_profesor_notas;                    -- ✓
UPDATE matricula SET nota_final = 8.75
WHERE id_alumno = 1 AND id_curso = 1;              -- ✓
UPDATE matricula SET fecha_matricula = '2026-01-01'
WHERE id_alumno = 1 AND id_curso = 1;              -- ✗
SELECT * FROM pago;                                -- ✗
RESET ROLE;

SET ROLE anon;
SELECT * FROM alumno;                              -- ✗
RESET ROLE;
```

Comprueba

Los ✗ deben dar `permission denied`. Si alguno funciona, has dado un permiso de más.

Como tú mismo (después del último `RESET ROLE`), `SELECT * FROM v_profesor_notas`; debe mostrar a Nerea con un 8.75 en Python.

Piensa

¿Por qué damos los permisos al **rol** y luego el rol a Lucía, en vez de dárselos directamente a Lucía? Imagina que la academia contrata a otra recepcionista.

¿Qué habría pasado en la prueba de `anon` si te hubieras saltado el paso 1?

¿Te has equivocado? Empieza de nuevo

Ejecuta esto y vuelve a lanzar tu código de esta práctica desde el principio. Solo borra lo de **esta** práctica: lo de las anteriores no se toca.

```
DROP VIEW IF EXISTS v_recepcion_alumnos, v_profesor_notas;
REVOKE ALL ON profesor, alumno, aula, curso, horario, matricula, pago
FROM rol_recepcion, rol_profesor;
DROP ROLE IF EXISTS lucia_recepcion, andres_profesor, rol_recepcion, rol_profesor;
```

Qué entregas

Captura de la lista de consultas con las de la práctica 6 (P6 · ...) y capturas de las pruebas del paso 5 donde se vean los ✓ y los X.

Anclaje curricular · RD 405/2023 · Módulo 0484 · RA2 · **criterios f, g y h**